

Chapter 1:

1. Introduction to System & Network Administration

Network administration is managing, monitoring, maintaining, and securing an organization's network. However, the actuality can be pretty complex. The process is typically handled by one or more administrators, who take care of all the specific tasks and procedures needed to ensure health and security, tasks that vary depending on the size and type of an organization.

To fully understand the scope of network administration and how an administrator's duties might vary from organization to organization, it's first necessary to understand the definition of a network. At its core, it's the system by which computers exchange information. However, it isn't merely made up of computers—networked computing devices exchange information via nodes established by cable or wireless media. And a network is composed of many different applications, devices (including computers, mobile phones, tablets, etc.), servers, routers, switches, and other hardware, not to mention tools like printers, fax machines, and email. Depending on the size of the system—from small organizations and personal LANs to enterprise webs existing globally—the needs of networks can vary greatly.

Within administration, there are five key areas:

Fault management

This area focuses on monitoring the entire infrastructure with the goal of identifying and addressing issues potentially affecting it. Fault management uses standard network protocols such as Simple Network Management Protocol (SNMP) to monitor infrastructure.

Configuration management

Configuration management focuses on tracking configuration and related changes affecting components, including switches, firewalls, hubs, and routers. Unforeseen changes can affect networks drastically, which can lead to downtime. Therefore, it's imperative to streamline, track, and manage configuration changes.

Account management

Usage can significantly affect many components of a network. Account management is used to track network utilization to estimate, prepare for, and potentially bill the usage of various departments within an organization. For smaller organizations, account management and billing may not be as important. But for all networks, monitoring utilization can aid in spotting specific trends and inefficiencies.

Performance management

For efficient operations, admins must focus on maintaining service levels, which is where performance management comes into play. Performance management is built around collecting various metrics and analytical data to continually assess network performance, including response times and packet loss.

Security management

One of the most essential areas of network administration, security management, aims to allow only authorized activity and authenticated devices and users access to the network. Security management utilizes several disciplines to maintain secure networks, including threat management, intrusion detection, and firewall management. Security management also encompasses the collection and analysis of relevant network information to detect and block malicious or suspicious activity.

1.1 Objectives/goals, challenges and common practices

Network Administration Goals/ Objectives:

While the size and needs of networks vary, the goals for admins are fairly standard. No matter the size of the system, administration nearly always aims to maintain a reliable, secure network conducive to business operations. Generally speaking, goals usually include the following:

- Maintaining resiliency and ensuring function and availability
- Planning and improving capacity to enable seamless access and operations
- Leveraging networking tools for automated administration and better control

- Tracking and documenting relevant changes
- Evaluating possible risks and orchestrating effective mitigations
- Preventing activities compromising or using the network as an attack vector
- Identifying and mitigating intrusions to avoid security breaches putting data at risk

Network Administration challenges

One of the biggest challenges that you face as a system administrator is protecting your network and systems from cyber-attacks. You have to constantly monitor and update your security policies, tools, and patches to prevent unauthorized access, data breaches, malware, ransomware, and other threats. You also have to educate your users on how to follow best practices and avoid risky behaviors. Additionally, you have to be prepared to respond quickly and effectively to any security incidents that may occur and minimize the damage.

Network Administration common practices

System administration isn't an easy job, but it's manageable with the right tools, the right people, and the right set of rules to live by. Learning some rules brings order out of the often chaotic world of system administration.

Best Practices for System Administrators

1. Be a Good Citizen

The first rule is a frequently overlooked one and a somewhat obscure one for a best practices list. However, its unusual inclusion makes it all the more compelling. Being a good citizen has to do with customer service. We don't think of network users as our customers, but they are exactly that.

For example, do your users see System Administrators as enablers and business assets or do they view them as sources of roadblocks or as production delays? Your job is to serve your users by maintaining systems, providing security, performing tasks within specified guidelines, and responding quickly to requests. Additionally, you're expected to do all of those things while maintaining a professional behavior with your users and your management.

2. Monitor Your Systems

Monitoring is more than simple UP/DOWN ping tests; it's a comprehensive insight into your environment that includes CPU, memory usage, network traffic, capacity, and environmental measurements. When you begin monitoring, you should collect statistics for your systems that establish a baseline of normal operating behavior to which you can refer in the future. You should collect usage statistics for CPU, memory, disk, and network. You also need to calculate growth statistics on logfiles, databases, and user data so that you can predict future capacity needs.

Gathering metrics, however, is only one aspect of monitoring. The other is alerting when those metrics fall outside of normal operating parameters. What happens when a filesystem fills up? Do you receive an alert at 85 percent capacity or does your system crash or experience a service outage due to a stopped process? Proactive alerting on system and service behavior is an essential part of your total datacenter picture.

3. Perform Disaster Recovery Planning

The third rule or best practice is to 'Perform Disaster Recovery Planning'. Contrary to some beliefs, disaster recovery doesn't necessarily mean recovery from a major disaster that affects the entire datacenter. It means recovery from any disaster, even single system disaster. One question you might consider as you think about disaster recovery is 'how are you going to fix the problem once it occurs?'. You might not have direct physical access to a failed system to help in its recovery. You'll have to rely on remote personnel working at the datacenter to recover a system that's experienced a hardware fault.

The other question to think about is 'where will you be when a disaster occurs?'. Disasters don't often occur at convenient times during working hours. They happen while you're away from the office and away from your computer. How will you meet the mean time to restore (MTTR) and the SLA for the failed system or systems when you have no access to them?

It's not enough to simply prepare for disasters; you have to plan for their occurrence. No amount of redundancy, load balancing, or regular backups will prevent disasters from happening. How to recover from a disaster, from a single system to an entire computing environment, is what you

have to think about and plan for. How you will connect to and recover those failed systems has to be part of the plan.

4. Document Everything

As challenging as it is, you must document standard procedures, connectivity information, regular maintenance tasks, and disaster recovery contingency plans.

Documentation is difficult because it requires the System Administrator to stop and move stepwise through each task, while thoughtfully documenting each procedure. It's time-consuming and labor-intensive to thoroughly document, take screenshots, describe procedures, and explain possible outcomes. If you don't have well documented procedures, then you'd better have the contingency plan of always being close to a computer and a network.

5. Establish Procedures for Your Work

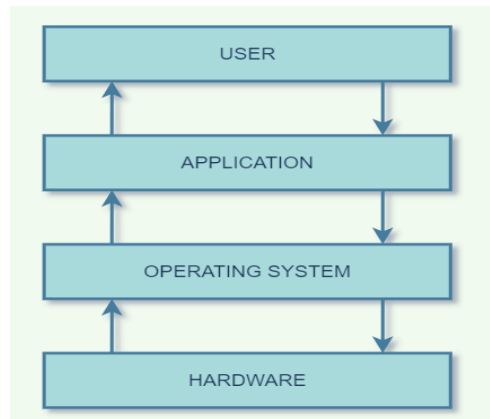
As you can surmise, rules four and five are closely related to each other. Establish standard procedures and document them. Standard procedures help you maintain consistency and reproducibility in your computing environment. Creating and adhering to a set of standard procedures has the added effect of stabilizing your systems and services, which, in turn, stabilizes your company's overall productivity.

System Administrators created these five best practices for System Administrators to use as guidelines that lead to more stable work environments and higher productivity. They'll help streamline your work, assist other System Administrators in your group, and maintain your sanity when things break.

1.2 Overview of an operating systems

An operating system (OS) manages all other applications and programs in a computer, and it is loaded into the computer by a boot program. It enables applications to interact with a computer's hardware. Through a designated application programme interface, the application programmes request services from the operating system (API). The kernel is the software that contains the

operating system's core components. To run other programmes, every computer has to have at least one operating system installed.



Operating Systems

Windows, Linux, and Android are examples of operating systems that enable the user to use programs like MS Office, Notepad, and games on the computer or mobile phone. It is necessary to have at least one operating system installed in the computer to run basic programs like browsers.

The core functions of an operating system include:

- **Managing hardware resources:** An operating system manages resources such as CPU, memory, and disk space, and assigns these resources to running applications.
- **Running applications:** An operating system provides an environment in which applications can run and interact with the user.
- **Providing a user interface:** An operating system provides a graphical user interface (GUI) that allows users to interact with the computer.
- Examples of popular operating systems include:
 - **Windows:** Microsoft Windows is the most popular desktop operating system, used by over 1 billion users worldwide. It has a wide range of features and applications, including the Office suite, gaming, and productivity tools.
 - **macOS:** macOS is the desktop operating system used by Apple Mac computers. It is known for its clean, user-friendly interface, and is popular among creative professionals.

- **Linux:** Linux is an **open-source** operating system that is available for free and can be customized to meet specific needs. It is used by developers and businesses, as well as individuals who prefer an open-source, customizable operating system.

Operating System Functions

- **Process Management:** An operating system manages the processor's work by allocating various jobs to it and ensuring that each process receives enough time from the processor to function properly.
- **Memory Management:** An operating system manages the allocation and de-allocation of the memory to various processes and ensures that the other process does not consume the memory allocated to one process.
- **Device Management:** There are various input and output devices. An OS controls the working of these input-output devices. It receives the requests from these devices, performs a specific task, and communicates back to the requesting process.
- **File Management:** An operating system keeps track of information regarding the creation, deletion, transfer, copy, and storage of files in an organized way. It also maintains the integrity of the data stored in these files, including the file directory structure, by protecting against unauthorized access.
- **Security:** The operating system provides various techniques which assure the integrity and confidentiality of user data. Following security measures are used to protect user data:
 - Protection against unauthorized access through login.
 - Protection against intrusion by keeping Firewall active.
 - Protecting the system memory against malicious access.
 - Displaying messages related to system vulnerabilities.
- **Error Detection:** From time to time, the operating system checks the system for any external threat or malicious software activity. It also checks the hardware for any type of damage. This process displays several alerts to the user so that the appropriate action can be taken against any damage caused to the system.
- **Job Scheduling:** In a multitasking OS where multiple programs run simultaneously, the operating system determines which applications should run in which order and how time should be allocated to each application.

Features of Operating Systems

- Here is a list of some important features of an operating systems:
 1. Provides a platform for running applications
 2. Handles memory management and CPU scheduling
 3. Provides file system abstraction
 4. Provides networking support
 5. Provides security features
 6. Provides user interface
 7. Provides utilities and system services
 8. Supports application development

1.3 Unix-like systems Vs Windows systems

Operating systems serve as the foundation for computer systems, enabling users to interact with hardware, run applications, and manage files.

What Is UNIX?

UNIX is a multitasking, multi-user operating system developed for workstations, servers, and other devices. Its numerous applications include database management, software development, and networked applications. Systems equipped with UNIX are preferred for their security, flexibility, and stability. This operating system is the basis for numerous others, including macOS and Linux.

UNIX was initially developed in the 1970s and was one of the first operating systems written using the C programming language. It is one of the most effective operating systems ever created, with numerous offshoots and a wide-reaching effect on the computing and electronics industries. Its main advantages include stability, interoperability, and portability across multiple heterogeneous environments and devices.

Today, UNIX and its variants are leveraged for multiple IT systems, including servers, workstations, mobile devices, embedded systems, and supercomputers. However, while Linux, macOS, and Android—arguably the most popular UNIX spin-offs—continue to maintain a strong presence in their respective markets, the demand for UNIX itself has declined.

One reason for this may be users moving away from reduced instruction set computer platforms and adopting x86-based alternatives capable of executing greater workloads and delivering higher performance more economically.

Enterprises will likely continue to leverage UNIX for mission-critical workloads; however, the operating system may continue to see falling demand due to a greater focus on IT modernization and consolidation. Naturally, this isn't happening immediately; UNIX is still the first preference for many IT teams regarding use cases such as vertical-specific software, cloud security, and data center application support.

Even if UNIX server sales **drop in the future**, its applications in the governance, telecom, and financial domains are expected to keep it alive and running. The complete abandonment of UNIX will likely be a long, slow journey.

What Is Linux?

Linux is an **open-source** operating system available **free of cost** and based on UNIX. It is widely leveraged across various devices for **stability**, **flexibility**, and **security**.

Linux is a popular operating system that is preferred for its reliability and security, performing better than its competitors in terms of **protection against viruses and malware**. It is also resistant to slowed-down performance, crashing, and expensive repairs and users need not pay licensing fees as often as they do for other commercial operating systems. Linux features a **zero cost** of entry and can be legally installed on any computer without any associated cost whatsoever.

Apart from its cost-effectiveness, Linux is an open-source system. The Linux kernel is among the world's most significant open-source projects, making it a benefit for enterprises seeking scalable growth without ballooning software costs. Most, if not all, DevOps lifecycles in enterprises around the globe rely on open-source software to reach their development goals. For instance, Linux makes it easy for developers to combine smaller software components they create with other software for building full-featured supply chains.

One significant way Linux helps bolster enterprise scalability is through containers, a popular way to deploy scalable enterprise applications. Containers help host applications on third-party cloud platforms. They simplify the automation of application scaling and help ensure that resource consumption matches the demand for application bandwidth at any given time.

Linux is the backbone of container operations. Users of Dockers and Kubernetes alike must be familiar with Linux. And this is just one of the many examples of Linux's flexibility. Other operating systems often lock the user into using software the 'right way' (according to the company backing it). Conversely, Linux users can make almost any change they desire at the platform level, allowing them to achieve business goals flexibly, swiftly, and accurately.

There isn't much that users cannot accomplish when they are familiar with Linux. This operating system grants access to thousands of tools and packages that support the development of robust services and systems.

What Is Windows?

Microsoft Windows is an operating system that features a **graphical user interface** and **compatibility** with a wide range of hardware and software, **primarily** for personal computers.

Every computer user has heard of Microsoft Windows, with over **75%** of desktop and laptop computers worldwide using this operating system. Available in **32-** and **64-bit** versions, Windows is a **user-friendly** operating system that features a graphical user interface (GUI), virtual memory management, support for multiple peripherals, and multitasking functionalities.

Windows **provides** both **server** and **client** versions. Popular client versions include Windows 98, ME, XP, Vista, 7, 8, and 10. Windows 11 is the newest version of this operating system, released in 2021. Server versions include NT Server, 2000 Server, 2003 Server, Server 2008 R2, Server 2016, and Server 2022 (latest version).

1.4 Linux distributions and UIs

A Linux distribution, commonly known as a **Linux distro**, refers to a complete operating system that is based on the **Linux kernel and bundled with various software packages**. Moreover, a Linux distro can be explained as follows:

A Linux distribution is a complete operating system that includes the **Linux kernel**, **system libraries**, **utilities**, **application software**, and a **package management** system. It is created by assembling various software components from different sources and packaging them together to provide a cohesive and user-friendly computing environment.

Main Characteristics and Components of a Linux Distribution

1. **Linux Kernel**: The Linux kernel serves as the core component of the operating system, providing low-level functionality, hardware abstraction, and device drivers.

2. **System Libraries:** Linux distributions include a set of system libraries, such as the GNU C Library (glibc), which provide essential functions and interfaces for applications to interact with the underlying operating system.
3. **User Interface:** Linux distributions offer different user interfaces, including graphical user interfaces (GUIs) like GNOME, KDE, or Xfce, as well as command-line interfaces (CLIs). These interfaces allow users to interact with the system and run applications.
4. **Software Packages:** Linux distributions come with a collection of software packages, including productivity tools, web browsers, email clients, media players, development tools, and more. These packages are typically managed and installed through a package management system.
5. **Package Management System:** Linux distributions utilize a package management system to install, update, and remove software packages.
6. **Configuration and Customization:** Linux distributions provide tools and utilities for configuring various aspects of the system, such as network settings, display preferences, user accounts, and security options. Users can customize the operating system to suit their specific needs and preferences.
7. **Support and Community:** Linux distributions are often backed by vibrant communities and support forums where users can seek assistance, share knowledge, and contribute to the development and improvement of the distribution.

Linux distributions provide to a wide range of users and purposes, from **general-purpose desktop** distributions like **Ubuntu** and **Fedora** to specialized distributions focused on specific tasks or environments, such as Kali Linux for penetration testing or CentOS for server deployments. Each distribution may have its own goals, target audience, default software selection, and release cycle, allowing users to choose the distribution that best fits their requirements and preferences.

Overall, Linux distributions offer a flexible, customizable, and open-source alternative to proprietary operating systems, empowering users with the ability to tailor their computing environment according to their specific needs.

1.5 Linux operations review

Linux is the **best-known** and **most-used** open source operating system. As an operating system, Linux is software that sits underneath all of the other software on a computer, receiving requests from those programs and relaying these requests to the computer's hardware.

In many ways, Linux is similar to other operating systems you may have used before, such as Windows, macOS (formerly OS X), or iOS. Like other operating systems, Linux has a graphical interface, and the same types of software you are accustomed to, such as word processors, photo editors, video editors, and so on. In many cases, a software's creator may have made a Linux version of the same program you use on other systems. In short: if you can use a computer or other electronic device, you can use Linux.

But Linux also is different from other operating systems in many important ways. First, and perhaps most importantly, Linux is **open source** software. The code used to create Linux is free and available to the public to view, edit, and for users with the appropriate skills to contribute to.

Linux is also different in that, although the core pieces of the Linux operating system are generally common, there are many **distributions** of Linux, which include different software options. This means that Linux is incredibly customizable, because not just applications, such as word processors and web browsers, can be swapped out. Linux users also can choose core components, such as which system displays graphics, and other user-interface components.

1.5.1 File system hierarchy and standard

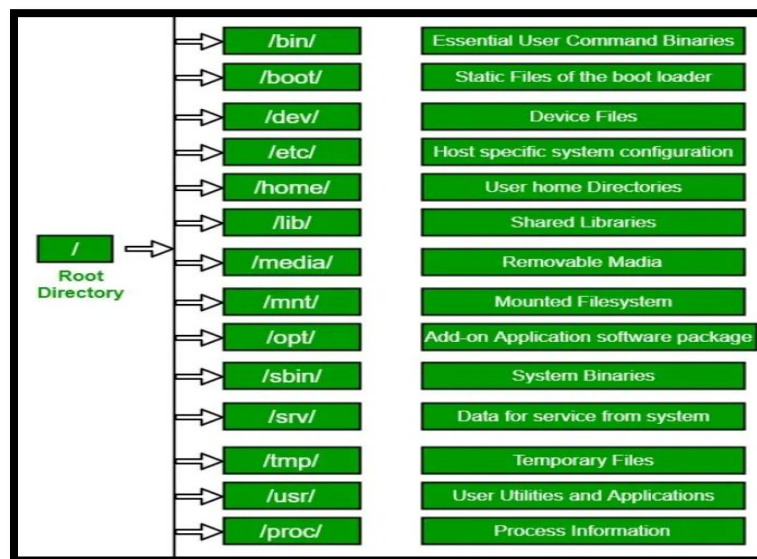
The Linux File Hierarchy Structure or the ***File system Hierarchy Standard*** (FHS) defines the directory structure and directory contents in Unix-like operating systems. It is maintained by the Linux Foundation.

1.5.1.1 Single-rooted hierarchy, seamless and extensible file systems

The root file system is the top of the hierarchical file tree. It contains the files and directories critical for system operation, including the device directory and programs for booting the system. The root file system also contains **mount points** where file systems can be mounted to connect to the root file system hierarchy.

In the FHS, all files and directories appear under the **root directory** /, even if they are stored on different physical or virtual devices.

Most of these directories exist in all UNIX operating systems and are generally used in much the same way; however, the descriptions here are those used specifically for the FHS and are not considered authoritative for platforms other than Linux.



Linux-directory

1.5.1.2 Mounting additional file systems

Before you access the files on a file system, you need to mount the file system. Mounting a file system attaches that file system to a directory (mount point) and makes it available to the system. The root (/) file system is always mounted. Any other file system can be connected or disconnected from the root (/) file system.

When you mount a file system, any files or directories in the underlying mount point directory are unavailable as long as the file system is mounted. These files are not permanently affected by the mounting process, and they become available again when the file system is unmounted. However, mount directories are typically empty, because you usually do not want to unclear existing files.

1.5.1.3 File system object oriented design and file system standard

A file is a collection of related information. The file system resides on secondary storage and provides efficient and convenient access to the disk by allowing data to be stored, located, and retrieved.

File system implementation in an operating system refers to how the file system manages the storage and retrieval of data on a physical storage device such as a hard drive, solid-state drive, or flash drive.

The file system implementation includes several components, including:

1. **File System Structure:** The file system structure refers to how the files and directories are organized and stored on the physical storage device.
2. **File Allocation:** The file allocation mechanism determines how files are allocated on the storage device. This can include allocation techniques such as contiguous allocation, linked allocation, indexed allocation, or a combination of these techniques.
3. **Data Retrieval:** The file system implementation determines how the data is read from and written to the physical storage device. This includes strategies such as buffering and caching to optimize file I/O performance.
4. **Security and Permissions:** The file system implementation includes features for managing file security and permissions. This includes access control lists (ACLs), file permissions, and ownership management.
5. **Recovery and Fault Tolerance:** The file system implementation includes features for recovering from system failures and maintaining data integrity. This includes techniques such as journaling and file system snapshots.

File system implementation is a critical aspect of an operating system as it directly impacts the performance, reliability, and security of the system. Different operating systems use different file system implementations based on the specific needs of the system and the intended use cases. Some common file systems used in operating systems include **NTFS** and **FAT** in Windows, and **ext4**, **HFS** and **ZFS** in Linux.

Implementation Issues

Management of disc space: To prevent space wastage and to guarantee that files can always be stored in contiguous blocks, file systems must manage disc space effectively. Free space management, fragmentation prevention, and garbage collection are methods for managing disc space.

Checking for consistency and repairing errors: The consistency and error-free operation of files and directories must be guaranteed by file systems. Journaling, check summing, and redundancy are methods for consistency checking and error recovery. File systems may need to perform recovery operations if errors happen in order to restore lost or damaged data.

Locking files and managing concurrency: To prevent conflicts and guarantee data integrity, file systems must control how many processes or users can access a file at once. File locking, semaphore, and other concurrency-controlling methods are available.

Performance optimization: File systems need to optimize performance by reducing file access times, increasing throughput, and minimizing system overhead. Caching, buffering, prefetching, and parallel processing are methods for improving performance.

Key Steps Involved In File System Implementation

File system implementation is a crucial component of an operating system, as it provides an interface between the user and the physical storage device. Here are the key steps involved in file system implementation:

1. **Partitioning the storage device:** The first step in file system implementation is to partition the physical storage device into one or more logical partitions. Each partition is formatted with a specific file system that defines the way files and directories are organized and stored.
2. **File system structures:** File system structures are the data structures used by the operating system to manage files and directories. Some of the key file system structures include the superblock, inode table, directory structure, and file allocation table.
3. **Allocation of storage space:** The file system must allocate storage space for each file and directory on the storage device. There are several methods for allocating storage space, including contiguous, linked, and indexed allocation.

4. **File operations:** The file system provides a set of operations that can be performed on files and directories, including create, delete, read, write, open, close, and seek. These operations are implemented using the file system structures and the storage allocation methods.
5. **File system security:** The file system must provide security mechanisms to protect files and directories from unauthorized access or modification. This can be done by setting file permissions, access control lists, or encryption.
6. **File system maintenance:** The file system must be maintained to ensure efficient and reliable operation. This includes tasks such as disk defragmentation, disk checking, and backup and recovery.

Overall, file system implementation is a complex and critical component of an operating system. The efficiency and reliability of the file system have a significant impact on the performance and stability of the entire system.

1.5.1.4 Unix file and directory permissions

Unix-like operating systems, such as Linux, running on shared high performance computers use settings called permissions to determine who can access and modify the files and directories stored in their file systems. Each file and directory in a file system is assigned "owner" and "group" attributes.

Most commonly, by default, the user who creates a file or directory is set as owner of that file or directory. When needed the system's root administrator can change the user attribute for files and directories.

The group designation can be used to grant teammates and/or collaborators shared access to an owner's files and directories, and provide a convenient way to grant access to multiple users.

File and directory permissions

In the output example below, the first character in each line indicates whether the listed object is a **file** or a **directory**. Directories are indicated by a (d); the absence of a d at the beginning of the first line indicates that myfile.txt is a regular file.

Example; -rw -r- -r - - abc

drwxr -xr -x bag

The letters **rw**x represent different permission levels:

Permission	Files	Directories
r	can read the file	can ls the directory
w	can write the file	can modify the directory's contents
x	can execute the file	can cd to the directory

Note the multiple instances of r, w, and x. These are grouped into three sets that represent different levels of ownership:

- **Owner or user permissions:** After the directory (**d**) slot, the first set of three characters indicate permission settings for the **owner** (also known as the user).

In the example **-rw-r--r--**, the owner permissions are **rw-**, indicating that the owner can read and write to the file but can't execute it as a program.

In the example `drwxr-xr-x`, the owner permissions are `rwX`, indicating that the owner can view, modify, and enter the directory.

- **Group permissions:** The second `rwX` set indicates the **group** permissions.

In the example `-rw-r--r--`, group members can only read the file.

In the example `drwxr-xr-x`, group members can view as well as enter the directory.

- **Other permissions:** The final `rwX` set is for "**other**". This is anyone outside the group. In both examples above, these are set to the same permissions as the group.

Changing file permissions

To change file and directory permissions, use the command `chmod` (change mode). The owner of a file can change the permissions for user (u), group (g), or others (o) by adding (+) or subtracting (-) the read, write, and execute permissions.

There are two basic ways of using `chmod` to change file permissions: The **symbolic** method and the **absolute** form.

Symbolic method

The first and probably easiest way is the relative (or symbolic) method, which lets you specify permissions with single letter abbreviations. A `chmod` command using this method consists of at least three parts from the following lists:

Access class	Operator	Access Type
u (user)	+ (add access)	r (read)
g (group)	- (remove access)	w (write)
o (other)	= (set exact access)	x (execute)

a (all: u, g, and o)

For example, To add permission for everyone to read a file in the current directory named `myfile`, is `chmod a+r myfile`

You can also specify multiple classes and types with a single command. For example, to remove read and write permission for group and other users (leaving only yourself with read and write permission) on a file named `myfile`, is `chmod go-rw myfile`

You can also specify that different permissions be added and removed in the same command. For example, to remove write permission and add execute for all users

on `myfile`, is `chmod a -w+x myfile`

Absolute form

The other way to use the `chmod` command is the absolute form, in which you specify a set of three numbers that together determine all the access classes and types. Rather than being able to change only particular attributes, you must specify the entire state of the file's permissions.

The three numbers are specified in the order: `user` (or owner), `group`, and `other`. Each number is the sum of values that specify read, write, and execute access:

Permission	Number
Read (r)	4
Write (w)	2
Execute (x)	1

Add the numbers of the permissions you want to give; for example:

- For file `myfile`, to grant read, write, and execute permissions to yourself ($4+2+1=7$), read and execute permissions to users in your group ($4+0+1=5$), and only execute permission to others ($0+0+1=1$), you would use: **`chmod 751 myfile`**.
- To grant read, write, and execute permissions on the current directory to yourself only, you would use: **`chmod 700 myfile`**

The three digit sequence as the sum of attributes you select from the following table:

Read by owner	400
Write by owner	200
Execute by owner	100
Read by group	040
Write by group	020
Execute by group	010
Read by others	004
Write by others	002
Execute by others	001

Sum all the accesses you wish to permit. For example, to give write and execute privileges to the owner of myfile (200+100=300), and give read privileges to all (400+040+004=444), you would enter: **chmod 744 myfile**

Some other examples are:

777	anyone can do anything (read, write, or execute)
755	you can do anything; others can only read and execute
711	you can do anything; others can only execute
644	you can read and write; others can only read

1.5.2 Essential shell commands

A shell is a **special user program** that provides an interface to the user to use operating system services. Shell accepts **human-readable** commands from the user and converts them into something which the kernel can understand. It is a command language interpreter that executes commands read from input devices such as keyboards or from files.

The shell gets started when the user logs in or starts the terminal.

Displaying the file contents on the terminal:

- **cat**: It is generally used to concatenate the files. It gives the output on the standard output.
- **more**: It is a filter for paging through text one screen full at a time.
- **less**: It is used to viewing the files instead of opening the file. Similar to *more* command but it allows backward as well as forward movement.
- **head**: Used to print the first N lines of a file. It accepts N as input and the default value of N is 10.
- **tail**: Used to print the last N-1 lines of a file. It accepts N as input and the default value of N is 10. etc.

1.5.2.1 Basic file manipulation commands and directory navigation commands

Here, we want to look at the commands used to manipulate files. We are going to **copy**, **move**, and **rename** files, plus a few commands for reading files in different ways.

1 **cp**: cp file1to file2

This command will copy the files and directories from the source path to the destination path. It can copy a file/directory with the new name to the destination path. It accepts the source file/directory and destination file/directory.

2 **mv**: mv file1to file2

Used to move files or directories. This command's working is almost similar to *cp* command but it deletes a copy of the file or directory from the source path.

- 3 **rm** : `rm [option] filename`, Used to remove files or directories.
- 4 **touch** : `touch filename`, Used to create or update a file. etc.

1.5.2.2 Advanced file manipulation commands (Init, processes, and threads)

The term file Manipulation refers to the process of **creating**, **modifying**, and **deleting** files and directories. In Linux, not only files but directories, devices, processes, sockets, and everything else is a file. All these files and directories are organized in a hierarchical tree-like structure, with the root directory ("/") being the topmost level.

This makes understanding basic file manipulation commands important for Linux users since it covers the way through which we can navigate our Linux systems from the command line.

Advanced commands provide greater customization and controls in more specialized situations once you become familiar with basic commands. Like:

Init is parent of all Linux processes with PID or process ID of 1. It is **the first process** to start when a computer boots up and runs until the system shuts down. `init` stands for **initialization**. In simple words the role of `init` is to create processes from script stored in the file `/etc/inittab` which is a configuration file which is to be used by initialization system. It is the last step of the kernel boot sequence.

`/etc/init tab` Specifies the **`init`** command control file.

- `init` script initializes the service. So, it responsible for initializing the system.
- `Init` scripts are also called `rc` scripts (run command scripts)
- `Init` script is also used in UNIX.

A **process** in Linux is nothing but a **program in execution**. It's a running instance of a program. Any command that you execute starts a process.

In Linux processes can be of two types:

- **Foreground Processes**
depend on the user for input
also referred to as interactive processes

- **Background Processes**

run independently of the user

referred to as non-interactive or automatic processes

A **thread** is a **lightweight** unit of execution **within a process** that can operate independently. It allows for concurrent execution of tasks, enabling parallel processing and efficient resource utilization. Threads **share the same memory space** and **resources** with other threads in the process, making data sharing and communication between threads seamless.

1.5.3 Advanced shell features

A shell is a program that acts as an interface to users, for the operating system. That is, it exposes the operating system to human users. These shells could either be command-line interfaces (CLI) or graphical user interfaces (GUI).

The Linux shell is a command-line interface that allows users to interact with the operating system and execute various built-in commands. It provides an interface between the user and the kernel to execute programs known as commands. In other words, the shell is the primary interface that provides users with a way to communicate with the Linux operating system at a more fundamental level than a GUI.

Some basic commands in the Linux shell include- ls, cd, mkdir, touch, cp, mv, rm, etc. With these basic commands, the Linux shell provides a wide range of advanced commands and features that allow users to perform more complex tasks. These include pipes, which allow users to connect the output of one command to the input of another command. And also redirection, which allows users to redirect input or output to a file.

There are many commands available in the Linux shell. Each command has its own set of options and arguments. Some of the popularly used lists of commands are as follows:

1. **cd**: Change directory. This command is used to change the current working directory.
2. **ls**: List directory contents. This command is used to list the files and directories in the current working directory.
3. **mkdir**: Make directory. This command is used to create a new directory.
4. **rm**: Remove file or directory. This command is used to delete files or current directories.
5. **cp**: Copy file or directory. This command is used to copy files or current directories.

6. `mv`: Move or rename a file or current directory. This command prompt is used to move files or directories to a new location or rename them.
7. `cat`: Concatenate and display files. This command is used to display the contents of an executable file.
8. `grep`: Search for a pattern in a file extension. This command is used to search for a specific string or pattern in a file type at the current time.
9. `pwd`: Print working directory. This simple command is used to display the current working directory.
10. `man`: Display manual pages. This command is used to display the manual pages for a specific command.

Overall, the Linux shell is a powerful tool that provides a text-based interface for users to interact with the operating system and has a complete control structure and environment variables. It allows users to perform various tasks more quickly and efficiently, provides greater flexibility and control, and can be used remotely. The Linux shell is based on a scripting language called Bash and provides a wide range of commands and features.